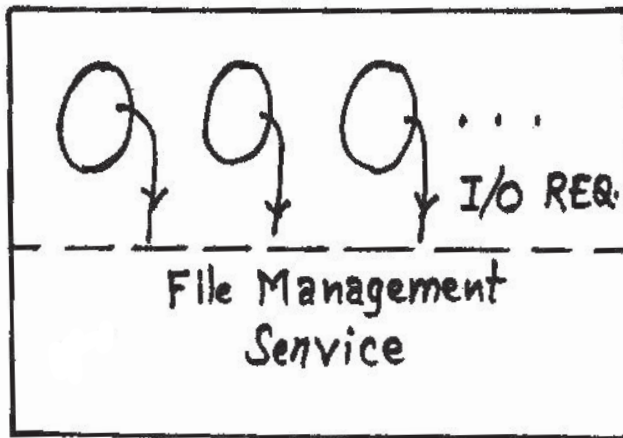


A PICTURE



Usual Service:

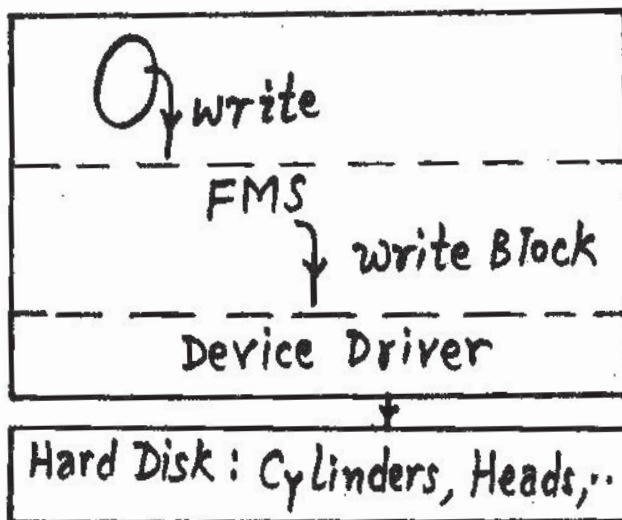
First Come
First Serve

FMS needs a different
style

Write Operations with concurrently running
processes.

sample Process A wants to write a 100 KB file
Assume Block 100,000 is free.

Write requests: B1k 100,000
B1k 100,001
: : :



Each write operation:
takes several msec

Write operations

Sample: Alongwith process A,
execute process B : reads a file

Requests : Blk 1000
 Blk 1001
 : : :

Worst Situation

Write Block 100,000
Read Block 1000
Write Block 100,001
Read Block 1001
Write Block 100,002
: : :

Satisfying requests in chronological order has problems.

Analogy: Courier Service

Customers

↓ ↓ ↓

Manager

↓ ↓

Office boy (courier)

Requests:

Letter to Jayanagar

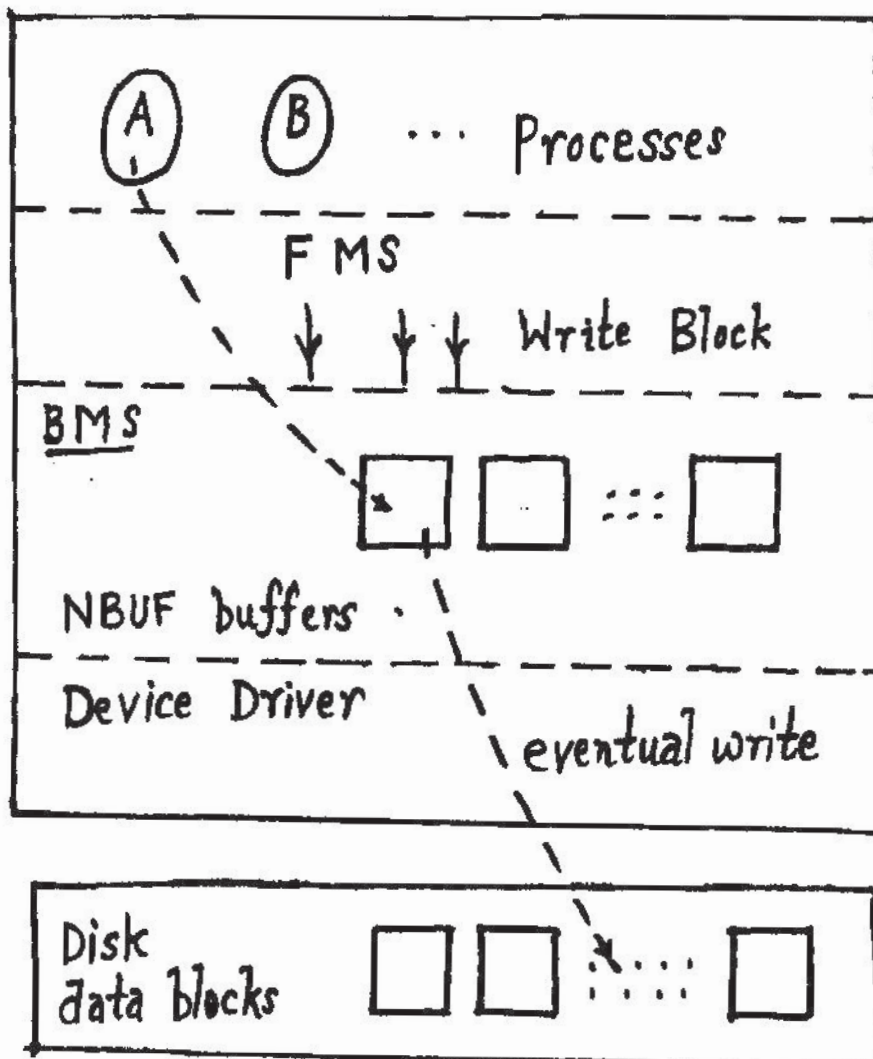
Letter to Vijayanagar

Letter to Jayanagar

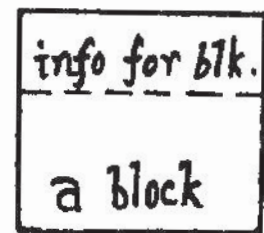
: : :

Solution: Hire a secretary : uses delaying tactics.

Write Operation



to be written
Cyl #, etc.



↓
struct buf
in
sys/buf.h

Buffer Management System uses delayed writes
so that more writes arrive.

Without delayed writes, hard disk life is effected ;
also, throughput is effected.

Eventual writes are done in ascending / descending
order of block numbers to reduce seeks.

Write operations

NBUF : tuneable parameter
usual values : 200, 500.

READ OPERATIONS

Datas to be transferred from data blocks on hard disk to process data space.

Consumption Rates : usually in bytes

Ex: an int : 4 bytes
OR a struct emp : 200 bytes
OR a double : 8 bytes

analogy : how we eat masala dosai

Catch: Block device driven : can only read a block at a time

i.e. Even for 4 bytes, device driver reads a block
Slightly later, process asks for next 4 bytes.

Question: should the driver read the same block again?

analogy: draw money from Bank.

read operation

Technique to be used: draw Rs. 500 & keep in pocket
go to bank when money exhausted
i.e. use a cache.

Use: buffered reads.

Keep a copy of entire block in one of the buffers & transfer the small number of bytes to process data space.

Question: When buffer cache is full
& a write request arrives:
which buffer to reuse for new block?

Buffers { to be written : dirty
read buffers

Analogy: Watch a dosai being eaten

- (a) most of the time, dosai will continue to disappear
 - (b) half dosai undisturbed for half an hour.
- i.e. accessed blocks continue to be accessed.

LRU algorithm

Hence select least recently used buffer for the new block.

If LRU buffer is clean, it can be simply reused; is dirty, a write must be scheduled before reuse.

Physical Writes: must be sorted as per block numbers.

POWER FAILURES

Like secretary getting into accident.

NOTE: BMS operations are transparent.

power failure: some blocks may never get written.

Sample: → created a file in the morning.

→ power failed in the evening.

i.e. whenever device driver is idle,

some writes may be scheduled.

Write is called SYNCHRONIZATION.

synchronization OR sync'ing
when to do?

(a) important edits: use sync(2) system call
OR sync(1) command

Use: sync ; sync ; sync ; sync

(b) application programmers : don't ask for sync.

daemon process : cron

periodically wakes up & asks for sync.

all these are generalized sync.

i.e. all dirty buffers are flushed.

(c) specialized sync

NOTE: write system call is apparently much faster
than read ! (just memory transfers)

(delayed reads are not possible!)

Usual writes : return immediately after
data transfer to buffer

Very important files: Audit Trail , Transaction Logs

Use: O_SYNC bit in open flags

synchronization

(c) specialized sync

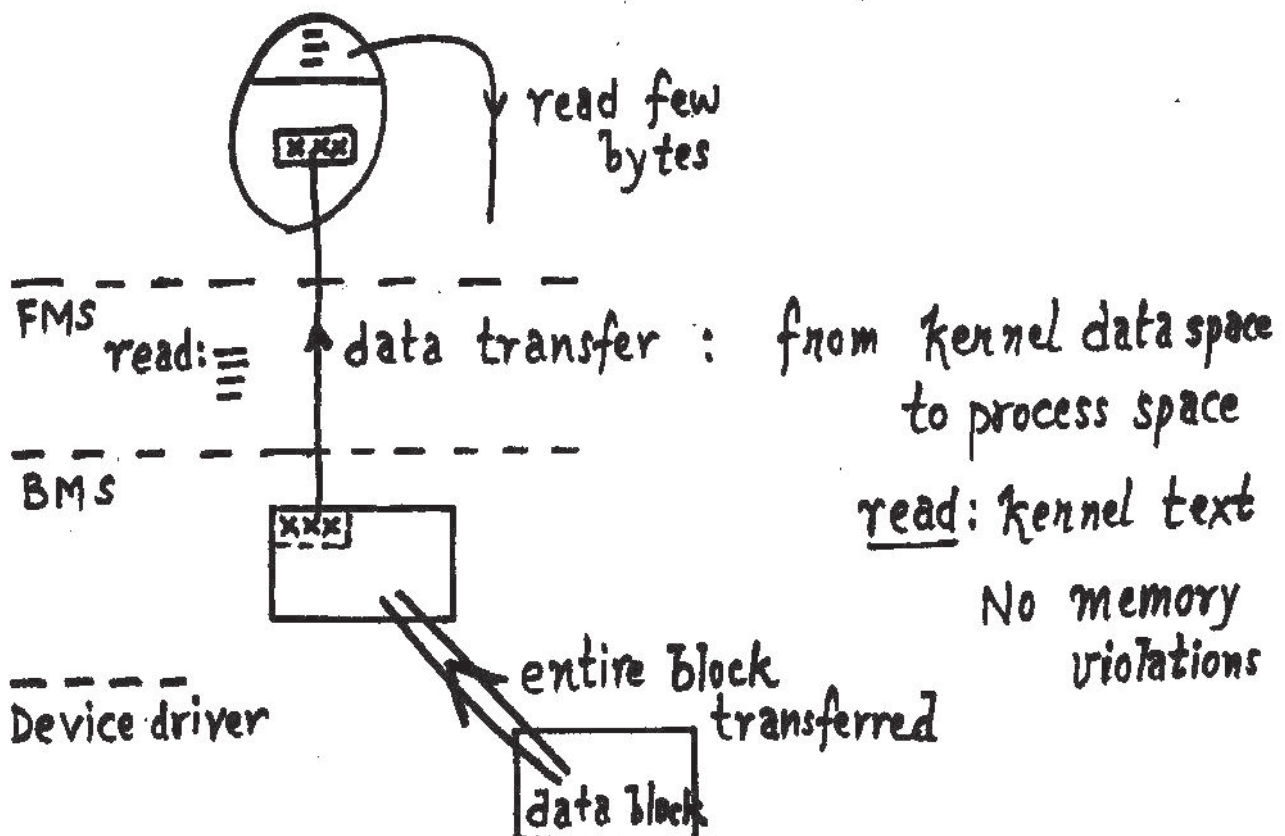
Open with `O_SYNC` : write returns only after physical write.

Too many `O_SYNC`'s : scheduled writes badly effected.

`O_SYNC` must be used sparingly.

SYSTEM CALL OVERHEADS

A problem with `read(2)` system call :



system call overheads

CPU modes {

- user : CPU executing process text
segment violation possible
- supervisory: CPU executing kernel text.
(privileged) mmu shuts up

any system call

CPU mode changed from user to supervisory
& eventually changed back to user mode.

Mode change time $\Rightarrow$ system call overhead

Sample process A reading 2/3 bytes at a time
lots of reads $\Rightarrow$ too much overhead.

Another problem: if other processes are also doing
lot of I/O.

buffer for process A may get replaced
 $\Rightarrow$ same data block may have to be
reread from hard disk.

NOTE: data transfer styles of processes
cannot be changed.

system call overheads

We need : a buffer above operating system.

analogy : Like Hindu Undivided Family
having cash cache.

C LIBRARY INTERFACE

for portability also

UNIX FMS : use read(2), write(2) system calls

DOS FMS : use int 21h, etc. system calls

i.e. each OS speaks a different
system call "language".

Ex: UNIX speaks "Kannada"

DOS speaks "Tamil"

AS400 speaks "Meghalayan"

Question : How to write portable programs?

ANS : Learn "English".

Programmer speaks : "English" OR C Library calls

These are translated to respective system calls.

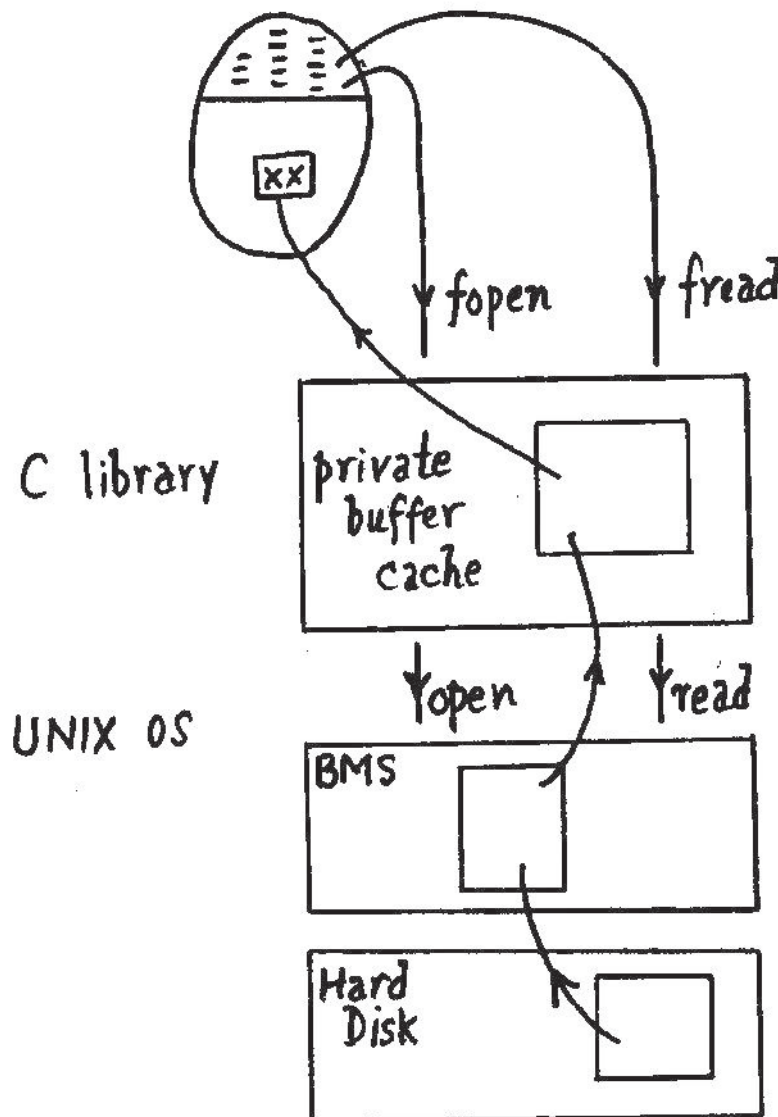
C library interface in libc.a

standard interface : stdio.h

Usually, most processes use C library calls

Reasons:

- (a) to have private buffer outside OS
- (b) to reduce system call overheads
- (c) to make programs portable



Manual pages

fopen (3)
 fread (3)
 fclose (3)
 fwrite (3)
 fseek (3)
 feof (3)
 ftell (3)
 fprintf (3)
 fscanf (3)
 ::

NOTE:
 triple data transfer

C Library interface

BMS buffer cache : still useful

Ex: data blocks of Vi command loaded only once from hard disk.

C library call : no system call overheads

Private Buffer

Note multi file accesses from a process.

Each file instantiation needs a buffer.

i.e. one buffer per fopen instantiation.

buffer not needed before fopen
& after fclose

hence: allocated in fopen
& freed in fclose

Buffer needs to contain: fd,
current file position,
etc.

in addition to a block.

Typedefined as FILE structure.

C library interface

FILE * : is used as fopen instantiation
(like fd used as open instantiation)
& is returned by fopen.

Ex: FILE * fp;
fp = fopen(---);

Formatted I/O

read(2), write(2) : unformatted operations

Most programs include `<stdio.h>`

NOTE: provide more, useful procedures as part
of C library

Ex: fprintf(3) fscanf(3)

stdio

Any process needs: standard input
standard output
standard error

stdin, stdout, stderr : are FILE *'s
printf => fprintf(stdout, ...)

C library interface stdio

standard file pointers (stdin, stdout, stderr)
can be used without "fopen"ing them.

other Facilities can be provided by C library.

Like: How to go to majestic?

Ex: feof(3), ftell(3)

Macros: getchar(3), putchar(3)

Hence

most programmers use C library most of
the time.

Also possible to specially change size of
the private buffer for
Kumbhakarn-style processes.

Exercises: can help us appreciate the
functionalities of the modules.

